

Decorator Pattern in Java File Reading

Java's `java.io` package is the **textbook real-world example** of the Decorator pattern. Instead of one class trying to do everything (read raw bytes + buffer + handle characters + parse lines), Java wraps simple objects with layers that each add one capability.

```
BufferedReader reader = new BufferedReader(  
    new InputStreamReader(  
        new FileInputStream("data.txt")));
```

- **FileInputStream** — the core component; just reads raw bytes.
- **InputStreamReader** — decorator that adds byte→character conversion.
- **BufferedReader** — decorator that adds buffering + `readLine()`.

Each wrapper implements the same interface as what it wraps, and **adds behavior without touching the original class**. That's the whole point of the pattern — open for extension, closed for modification.

Template Method

```
abstract class Beverage {  
  
    // Template Method  
    public final void prepareRecipe() {  
        boilWater();  
        brew();  
        pourInCup();  
        addCondiments();  
    }  
  
    private void boilWater() {  
        System.out.println("Boiling water");  
    }  
  
    abstract void brew();  
  
    private void pourInCup() {  
        System.out.println("Pouring into cup");  
    }  
  
    abstract void addCondiments();  
}
```

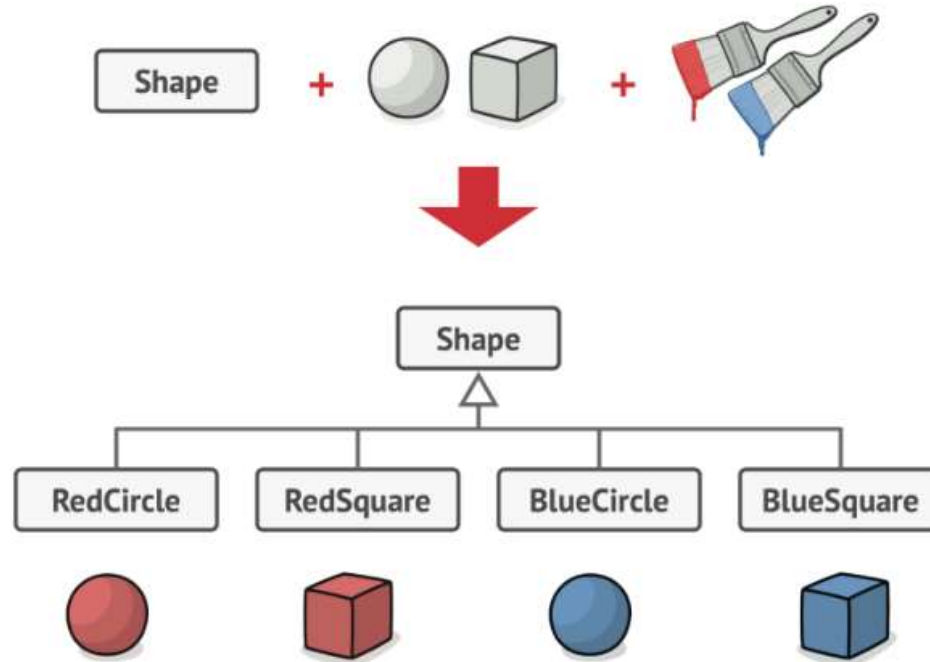
```
public final void prepareRecipe() /  
private void boilWater() /  
private void pourInCup() /  
protected abstract void brew(); /  
protected abstract void addCondiments
```

Template Method

Standard way

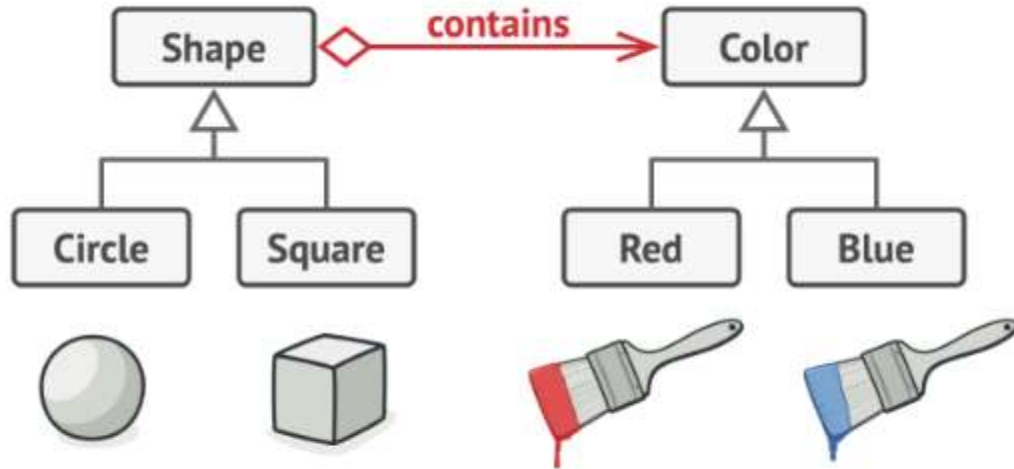
```
public final void prepareRecipe()    // fixed algorithm
private void boilWater()             // fixed step
private void pourInCup()             // fixed step
protected abstract void brew();      // customizable
protected abstract void addCondiments(); // customizable
```

Bridge Pattern



Number of class combinations grows in geometric progression.

Bridge Pattern



You can prevent the explosion of a class hierarchy by transforming it into several related hierarchies.

Bridge Pattern

- separates an abstraction from its implementation so that both can vary independently.
- useful when you have two independent dimensions of variation. Instead of creating subclasses for every combination, you connect the two hierarchies using composition.
- Example

Suppose your application can send notifications through different channels.

Abstraction

- Alert Notification
- Promotion Notification
- OTP Notification

Implementation

- Email
- SMS
- Push Notification

AlertEmail

AlertSMS

AlertPush

PromotionEmail

PromotionSMS

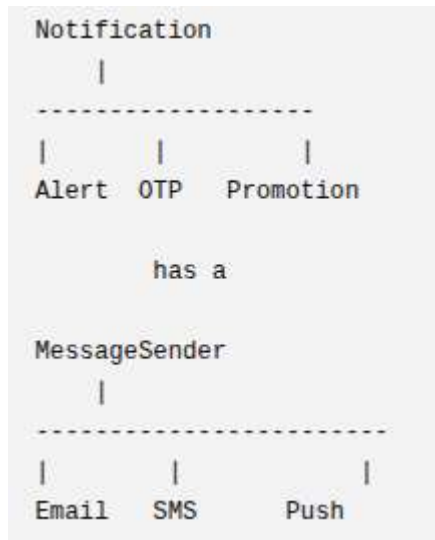
PromotionPush

OTPEmail

OTPSMS

OTPPush

With Bridge



```
interface MessageSender {
    void send(String msg);
}

class EmailSender implements
MessageSender {
    public void send(String msg){
        System.out.println("Email: " +
msg);
    }
}

.....
```

```
abstract class Notification {
    protected MessageSender sender;

    Notification(MessageSender sender){
        this.sender = sender;
    }

    abstract void notifyUser();
}

class AlertNotification extends Notification {

    AlertNotification(MessageSender
sender){
        super(sender);
    }

    void notifyUser(){
        sender.send("Server is down!");
    }
}

...
```